Flujo de autenticación

Space Trader Back · guía para desarrolladores de front y back

Fuente de verdad: doc/authentication.md y doc/architecture.md. Este PDF es un resumen visual.

1. Visión general

El back es un **Backend for Frontend (BFF)**: el front (SPA Vue) solo habla con este back y nunca con SpaceTraders. La persona usuaria se autentica contra el back con email y contraseña. Cada agente de SpaceTraders vinculado tiene su propio token, que el back guarda cifrado y el front nunca ve.

La sesión se apoya en dos tokens con papeles distintos: un **access token** de vida corta, que acompaña cada petición, y un **refresh token** de vida larga, que solo sirve para conseguir access tokens nuevos.

- **(1)** HTTPS entre orígenes distintos. CORS admite credenciales solo para orígenes permitidos, así que la cookie del refresh token viaja en `/auth/login`, `/auth/refresh` y `/auth/logout`.
- **(2)** El back llama a SpaceTraders con el token del agente, que nunca sale del back.

2. Conceptos

2.1 Access token

- **Qué es:** un JWT (JSON Web Token) firmado por el back con RS256: clave privada para firmar, clave pública para verificar.
- **Para qué sirve:** demuestra quién eres en cada petición protegida. El front lo envía en la cabecera `Authorization: Bearer <token>`.
- **Dónde vive:** solo en la memoria del front, nunca en `localStorage` ni en cookies. Si la pestaña se recarga se pierde, y se recupera con el refresh token.
- **En el servidor:** no se guarda. El back solo comprueba la firma y la caducidad (es *stateless*). Por eso un access token emitido no se puede revocar: vale hasta que caduca.
- **Duración:** 15 minutos. Es corta a propósito: como no se puede revocar y el refresh token no rota, es el único límite que tiene un access token robado.

2.2 Refresh token

- **Qué es:** una cadena aleatoria opaca de 256 bits, sin información dentro.
- **Para qué sirve:** sirve únicamente para pedir un access token nuevo sin volver a introducir la contraseña. Representa **una sesión**: un login en un dispositivo.

- **Dónde vive en el cliente:** en una cookie `httpOnly; Secure; SameSite=Strict; Path=/auth`.
- `httpOnly`: JavaScript no puede leerla, lo que la protege de XSS. `Secure`: solo viaja por HTTPS (los navegadores tratan `localhost` como seguro). `SameSite=Strict`: no se envía en peticiones iniciadas desde otros sites, lo que la protege de CSRF. `Path=/auth`: solo llega a login, refresh y logout.
- **Dónde vive en el servidor:** en base de datos se guarda solo su hash SHA-256, nunca el valor en claro. Una filtración de la BD no permite suplantar sesiones.
- **Duración:** 24 horas o 30 días desde el login, según el modo elegido (apartado 2.4). Nunca se alarga con el uso.
- **Sin rotación:** cada refresh valida el mismo token y no emite uno nuevo. Es una simplificación aceptada: un refresh token robado vale hasta que caduca o se cierra esa sesión.

2.3 ¿Por qué dos tokens?

	Access token	Refresh token
Se usa en	Cada petición a la API	Solo /auth/login, /auth/refresh y /auth/logout
Duración	Corta: 15 min	Larga: 24 h o 30 días
Revocable	No, caduca solo	Sí, está en la BD
Accesible desde JS	Sí (memoria)	No (cookie httpOnly)
Formato	JWT firmado, autocontenido	Cadena opaca; el servidor guarda su hash

El access token se usa constantemente y no se puede revocar, así que dura poco. El refresh token viaja pocas veces, está mejor protegido y se puede revocar, así que puede durar mucho.

2.4 Modos de sesión: «recordarme»

El login acepta un campo opcional `rememberMe` (booleano, `false` si no se envía). Persistir la sesión es siempre una elección explícita: los clientes que no envían nada no dejan sesiones de 30 días abiertas.

Modo	Cookie en el navegador	Validez en el servidor	Pensado para
rememberMe: true	Persistente, con Expires = login + 30 días. Sobrevive al cierre del navegador.	30 días desde el login	Equipo personal
rememberMe: false o ausente (por defecto)	De sesión, sin Expires: el navegador la descarta al cerrarse.	24 horas desde el login	Equipo compartido o uso puntual

- **¿Por qué el límite de 24 h si la cookie ya es de sesión?** Los navegadores pueden restaurar las cookies de sesión al reabrir pestañas, así que la cookie podría vivir días. El servidor pone un tope que no depende del navegador.
- **El modo no se guarda aparte:** la caducidad se fija al emitir el refresh token y queda en la BD. `/auth/refresh` no vuelve a enviar la cookie, así que una cookie de sesión nunca se convierte en persistente.
- Un `rememberMe` que no sea booleano (por ejemplo `"true"` como texto, o `1`) se rechaza con `400 http_error`, igual que un `email` o `password` que no sean texto, y no se inicia sesión.

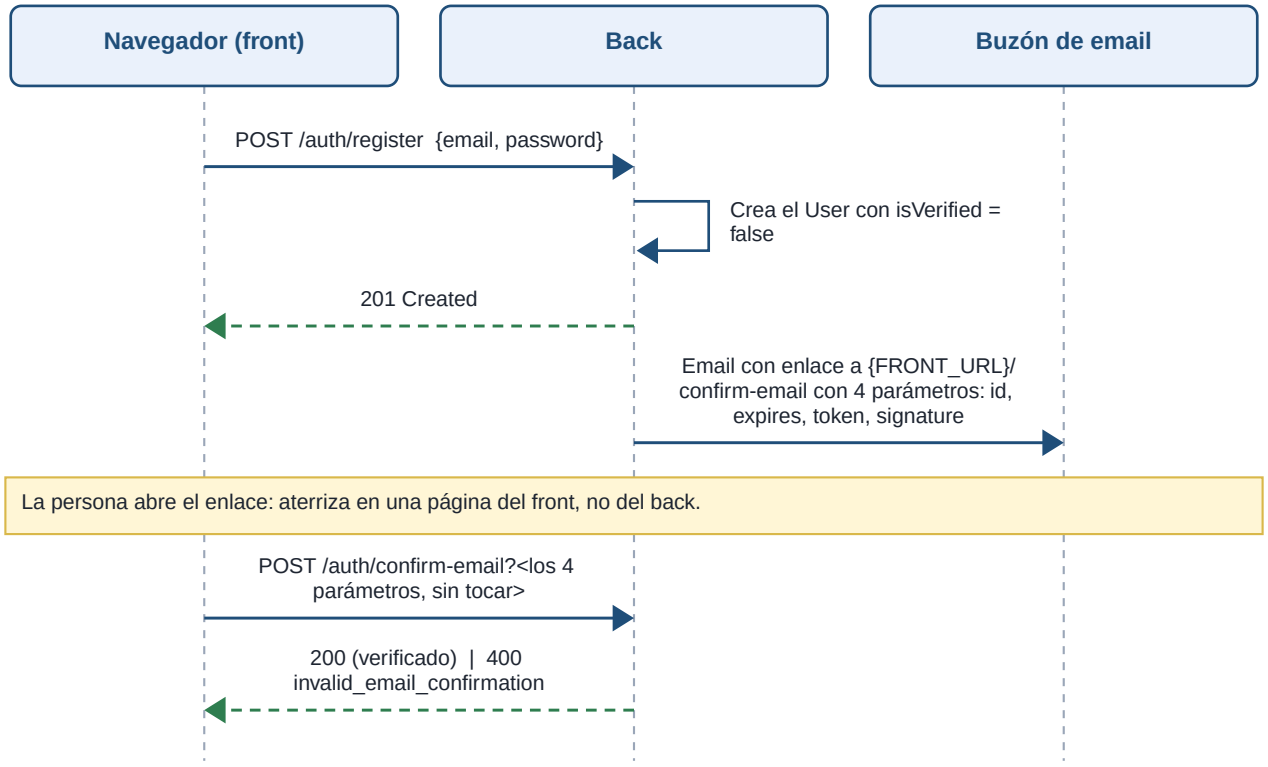
2.5 CORS con credenciales

El front y el back se sirven desde **orígenes distintos** (en desarrollo, localhost con puertos distintos). Para que el navegador guarde la cookie que devuelve `/auth/login` y la envíe en `/auth/refresh` y `/auth/logout`, hacen falta dos cosas:

- **En el front:** las llamadas a `/auth/*` se hacen con credenciales (`fetch(..., { credentials: 'include' })`, o `withCredentials: true` en axios).
- **En el back:** responde con `Access-Control-Allow-Credentials: true` y con `Access-Control-Allow-Origin` igual al origen concreto que hace la petición, nunca `*`, y solo si está entre los orígenes permitidos (variable `CORS_ALLOW_ORIGIN`).
- Como front y back comparten site (mismo host, distinto puerto), `SameSite=Strict` no impide el envío de la cookie. Si en producción viven en sites distintos, habrá que revisarlo (pendiente en doc/architecture.md).

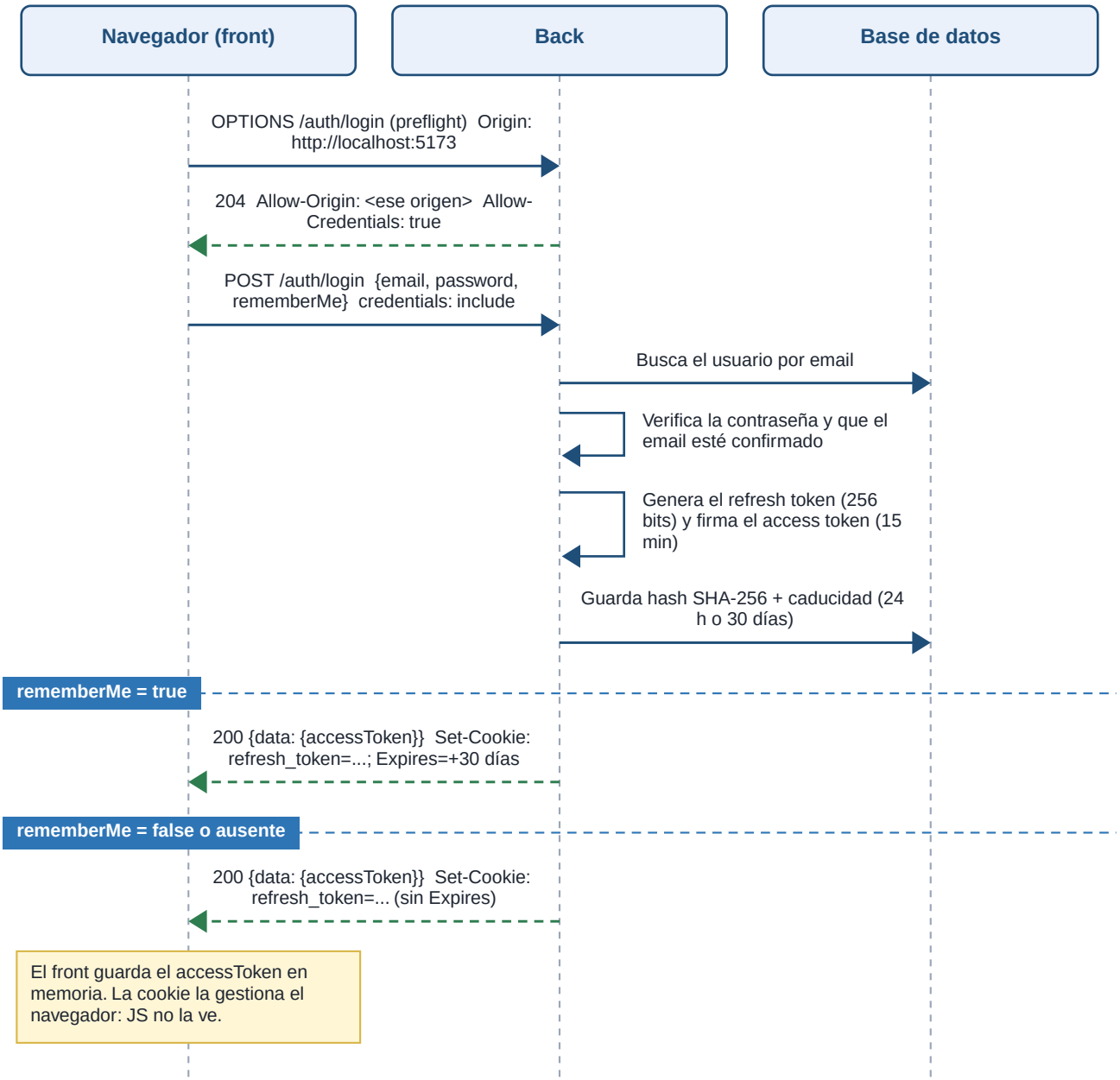
3. Flujos

3.1 Registro y verificación de email



- Un email ya registrado devuelve `409 email_already_registered`.
- El 400 de confirmación tiene el mismo cuerpo falte un parámetro, sobre uno, esté alterado o caducado: el back no revela qué ha fallado.
- `POST /auth/resend-confirmation` reenvía el email y responde `202`.

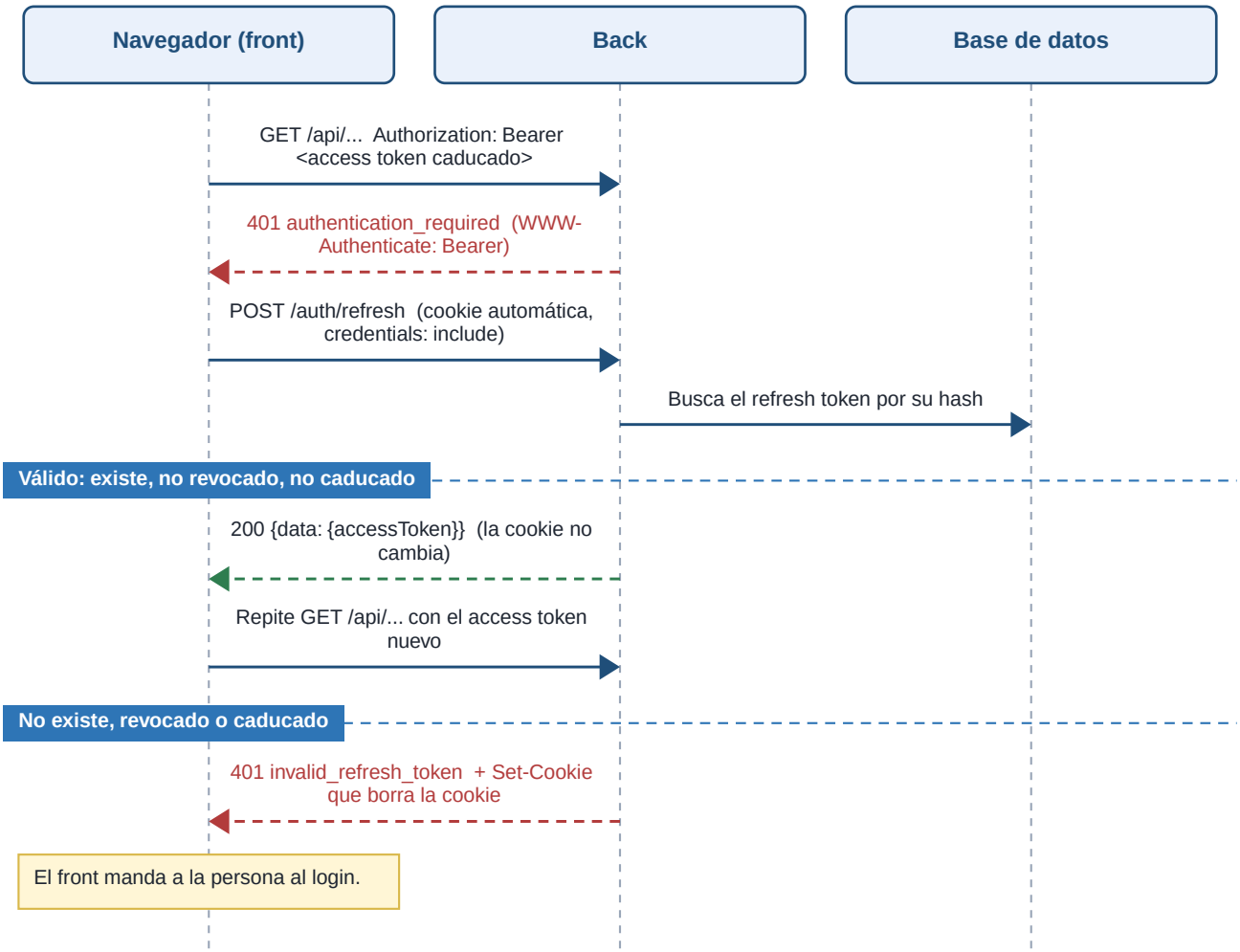
3.2 Login



Errores del login:

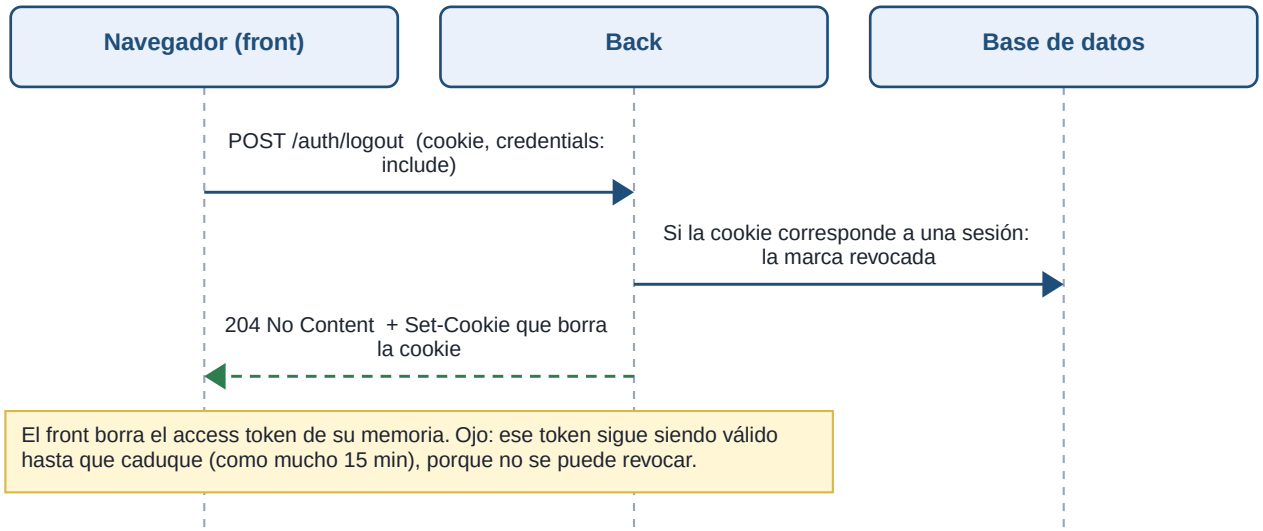
- `401 invalid_credentials`: contraseña incorrecta o email inexistente, con el mismo cuerpo en ambos casos para no revelar qué emails existen. Es un error del formulario: el front **no** debe intentar un refresh.
- `403 email_not_verified`: las credenciales son correctas pero el email no está confirmado. Solo se devuelve con la contraseña correcta.
- `400 http_error`: body mal formado; por ejemplo, `rememberMe` no es booleano o `email` no es texto.
- Cada login crea una sesión nueva e independiente: una persona puede tener varias a la vez (móvil, portátil...).

3.3 Peticiones autenticadas y renovación



- **Regla del front:** ante un `401 authentication_required`, intenta `/auth/refresh` **una sola vez**. Si funciona, repite la petición original; si falla, al login.
- El back responde `authentication_required` igual si el JWT falta, está mal formado, ha caducado o es de un usuario que ya no existe.
- Al recargar la pestaña el access token se pierde: el front llama a `/auth/refresh` al arrancar para recuperar la sesión, si la cookie sigue viva.

3.4 Logout

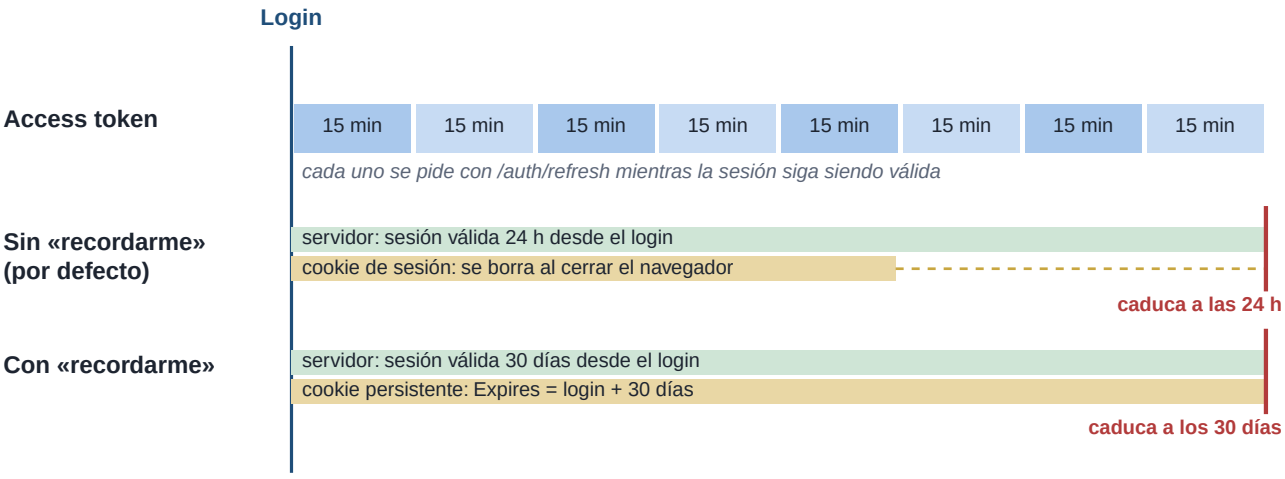


- Revoca **solo esa sesión**; las de otros dispositivos siguen activas.
- No necesita access token: la cookie basta para identificar la sesión.
- Es idempotente: sin cookie o con una cookie desconocida responde igualmente `204`.

3.5 Reset del universo de SpaceTraders

SpaceTraders reinicia su universo periódicamente y los tokens de agente dejan de valer. Cuando el back lo detecta, revoca **todas** las sesiones activas del usuario afectado, de los dos modos, y el siguiente `/auth/refresh` de cada dispositivo devuelve `401 invalid_refresh_token`.

3.6 Línea temporal de una sesión



Escala ilustrativa: cada fila tiene su propia escala. Ninguna duración se alarga con el uso; todas cuentan desde el login.

4. Referencia rápida

4.1 Endpoints de autenticación

Endpoint	Requiere	Éxito	Errores
POST /auth/register	Nada	201	409 email_already_registered; 422 validation_failed
POST /auth/confirm-email	Nada	200	400 invalid_email_confirmation

Endpoint	Requiere	Éxito	Errores
POST /auth/resend-confirmation	Nada	202	422 validation_failed
POST /auth/login	Nada	200 + Set-Cookie	401 invalid_credentials; 403 email_not_verified; 400 http_error
POST /auth/refresh	Cookie	200	401 invalid_refresh_token (borra la cookie)
POST /auth/logout	Cookie (opcional)	204 + borra la cookie	Ninguno (idempotente)
Resto de /api/...	Bearer JWT	Según endpoint	401 authentication_required

Todos los errores usan el envelope común del back (ver doc/errors.md).

4.2 Firewall (Symfony Security)

- `login` (`^/auth/login$`): *stateless*, con `json_login`. Un *user checker* exige el email verificado y unos handlers propios construyen la respuesta de éxito y de fallo.
- `main`: *stateless*, con el autenticador JWT de LexikJWTAuthenticationBundle.
- Rutas públicas: `/auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `/auth/confirm-email`, `/auth/resend-confirmation` y `/api/doc`. El resto exige `ROLE_USER`.

4.3 Dónde mirar en el código

- `config/packages/security.yaml`: firewalls y reglas de acceso.
- `config/packages/lexik_jwt_authentication.yaml`: claves y duración del access token.
- `config/packages/nelmio_cors.yaml`: CORS y credenciales.
- `src/Security/`: emisión y verificación del refresh token, la cookie y los handlers del login.
- `src/Controller/Auth/`: endpoints de registro, confirmación, login, refresh y logout.